

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

ОТЧЕТ
О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ №1
ПО ДИСЦИПЛИНЕ
«ОСНОВЫ КОМПЬЮТЕРНОГО МОДЕЛИРОВАНИЯ
ДИНАМИЧЕСКИХ СИСТЕМ»
ВАРИАНТ ЗАДАНИЯ №20

Выполнил студент группы М8О-206Б-23
Сергеев В. С. _____

подпись, дата

Проверил и принял

Ст. преп. каф. 802 Волков Е.В. _____

подпись, дата

с оценкой _____

Вариант №20

Задание:

Построить заданную траекторию и анимацию движения точки, а также отобразить стрелки радиус-вектора, скорости и ускорения. Построить радиус кривизны траектории.

Закон движения точки:

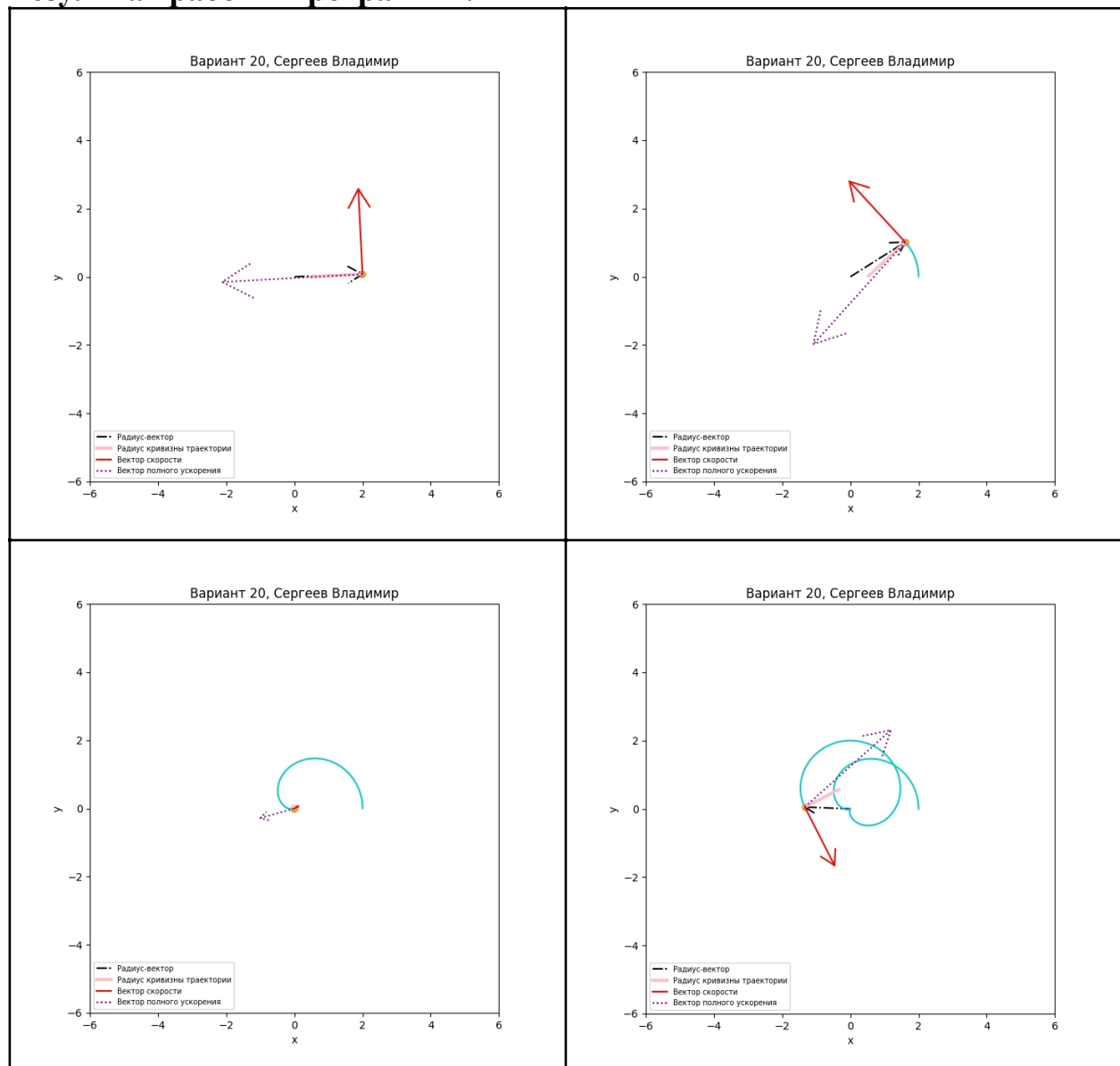
$$r(t) = 1 + \cos t$$

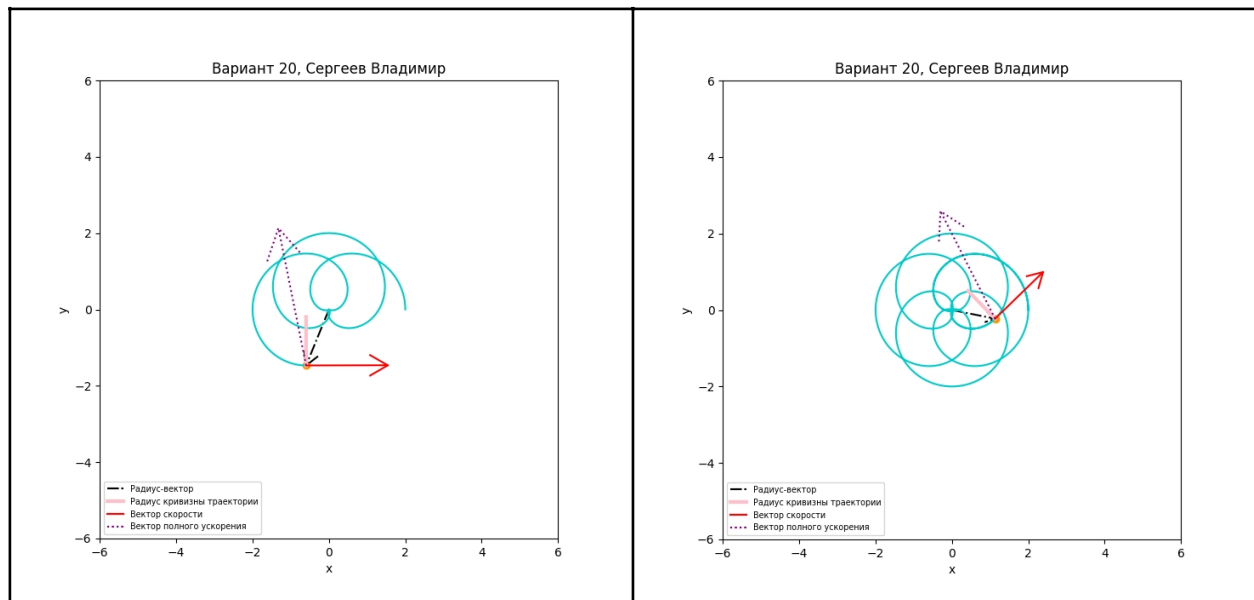
$$\varphi(t) = 1.25t$$

Текст программы

Текст программы в приложении №1.

Результат работы программы:





Вывод:

В ходе выполнения лабораторной работы была написана программа на языке Python, для симуляции и визуализации движения точечного тела в соответствии с заданными законами движения.

Приложение №1

```
import numpy as np
import sympy as sp
from matplotlib import pyplot as plt
from matplotlib.animation import FuncAnimation

class DisplayGroup:
    def __init__(self):
        if displaying_objects_switches['Position']: # Point particle
            self.particle, = region.plot(np.array([]), np.array([]), color='ff9600', marker='o')
        if displaying_objects_switches['Position vector']: # Position vector
            self.position_arrow = Arrow(color='black', line_style='-.')
        if displaying_objects_switches['Trajectory']: # Trajectory
            self.curve, = region.plot(np.array([]), np.array([]), color='#00cccc')
        if displaying_objects_switches['Curvature radius']: # Radius of curvature
            self.curvature_radius, = region.plot(np.array([]), np.array([]), color='pink', lw=3)
        if displaying_objects_switches['Velocity vector']: # Velocity vector
            self.velocity_arrow = Arrow(color='red')
        if displaying_objects_switches['Total acceleration vector']: # Acceleration vector
            self.acceleration_arrow = Arrow(color='purple', line_style=':')
        if displaying_objects_switches['Tangential acceleration vector']: # Tangential acceleration vector
            self.tangential_acceleration_arrow = Arrow(color='#228b22', line_style='--')
        if displaying_objects_switches['Centripetal acceleration vector']: # Centripetal acceleration vector
            self.centripetal_acceleration_arrow = Arrow(color='blue', line_style='--')

    def update(self, x, y, velocity_x, velocity_y, acceleration_x, acceleration_y, tangential_x, tangential_y,
               centripetal_x, centripetal_y, curvature_radius_x, curvature_radius_y):

        if displaying_objects_switches['Position']:
            self.particle.set_data(np.array([x[-1]]), np.array([y[-1]]))
        if displaying_objects_switches['Position vector']:
            self.position_arrow.update(0, 0, x[-1], y[-1])
        if displaying_objects_switches['Trajectory']:
            self.curve.set_data(x, y)
        if displaying_objects_switches['Curvature radius']:
            self.curvature_radius.set_data(np.array([x[-1], x[-1] + curvature_radius_x]),
                                           np.array([y[-1], y[-1] + curvature_radius_y]))
        if displaying_objects_switches['Velocity vector']:
            self.velocity_arrow.update(x[-1], y[-1], x[-1] + velocity_x, y[-1] + velocity_y)
        if displaying_objects_switches['Total acceleration vector']:
            self.acceleration_arrow.update(x[-1], y[-1], x[-1] + acceleration_x, y[-1] + acceleration_y)
        if displaying_objects_switches['Tangential acceleration vector']:
            self.tangential_acceleration_arrow.update(x[-1], y[-1], x[-1] + tangential_x, y[-1] + tangential_y)
        if displaying_objects_switches['Centripetal acceleration vector']:
            self.centripetal_acceleration_arrow.update(x[-1], y[-1], x[-1] + centripetal_x, y[-1] + centripetal_y)

    def return_plots(self):
        plots_for_return = []

        if displaying_objects_switches['Position']:
            plots_for_return.append(self.particle)
        if displaying_objects_switches['Position vector']:
            plots_for_return += [self.position_arrow.return_plot()]
        if displaying_objects_switches['Trajectory']:
            plots_for_return.append(self.curve)
        if displaying_objects_switches['Curvature radius']:
            plots_for_return.append(self.curvature_radius)
        if displaying_objects_switches['Velocity vector']:
            plots_for_return += [self.velocity_arrow.return_plot()]
        if displaying_objects_switches['Total acceleration vector']:
            plots_for_return += [self.acceleration_arrow.return_plot()]
        if displaying_objects_switches['Tangential acceleration vector']:
            plots_for_return += [self.tangential_acceleration_arrow.return_plot()]
        if displaying_objects_switches['Centripetal acceleration vector']:
            plots_for_return += [self.centripetal_acceleration_arrow.return_plot()]

        return plots_for_return

    def return_plots_for_legend(self):
        plots_for_return = []

        if displaying_objects_switches['Position vector']:
            plots_for_return += [self.position_arrow.return_plot()]
        if displaying_objects_switches['Curvature radius']:
            plots_for_return.append(self.curvature_radius)
        if displaying_objects_switches['Velocity vector']:
            plots_for_return += [self.velocity_arrow.return_plot()]
```

```

        if displaying_objects_switches['Total acceleration vector']:
            plots_for_return += [self.acceleration_arrow.return_plot()]
        if displaying_objects_switches['Tangential acceleration vector']:
            plots_for_return += [self.tangential_acceleration_arrow.return_plot()]
        if displaying_objects_switches['Centripetal acceleration vector']:
            plots_for_return += [self.centripetal_acceleration_arrow.return_plot()]

    return plots_for_return

@staticmethod
def return_names_for_legend():
    names_for_return = []

    if displaying_objects_switches['Position vector']:
        names_for_return.append('Радиус-вектор')
    if displaying_objects_switches['Curvature radius']:
        names_for_return.append('Радиус кривизны траектории')
    if displaying_objects_switches['Velocity vector']:
        names_for_return.append('Вектор скорости')
    if displaying_objects_switches['Total acceleration vector']:
        names_for_return.append('Вектор полного ускорения')
    if displaying_objects_switches['Tangential acceleration vector']:
        names_for_return.append('Вектор тангенциального ускорения')
    if displaying_objects_switches['Centripetal acceleration vector']:
        names_for_return.append('Вектор нормального ускорения')

    return names_for_return

class Arrow:
    """Class for vectors visualization."""

    def __init__(self, color='black', line_style='-'):
        self.arrow_template_radaii = np.array([0.25, 0, 0.25])
        self.arrow_template_angles = np.array([np.pi * 5 / 6, 0, np.pi * 7 / 6])
        self.body, = region.plot(np.array([]), np.array([]), color=color, linestyle=line_style)
        self.head, = region.plot(np.array([]), np.array([]), color=color, linestyle=line_style)

    def update(self, x1, y1, x2, y2):
        self.body.set_data(np.array([x1, x2]), np.array([y1, y2]))
        angle = np.arctan2(y2 - y1, x2 - x1)
        length = np.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
        head_x, head_y = polar_coordinates_to_cartesian(
            self.arrow_template_radaii * length,
            self.arrow_template_angles + np.full(3, angle)
        )
        self.head.set_data(np.full(3, x2) + head_x, np.full(3, y2) + head_y)

    def return_plot(self):
        return self.body, self.head

def calculate_values(expression: sp.Expr, time_points: np.ndarray) -> np.ndarray:
    """Calculates values of given expression at given time points."""
    values = sp.lambdify(t, expression, 'numpy')(time_points)
    if not isinstance(values, np.ndarray):
        values = np.full(time_points.size, values)
    return values

def polar_coordinates_to_cartesian(r, fi):
    """Converts polar coordinates to cartesian coordinates."""
    x = r * np.cos(fi)
    y = r * np.sin(fi)
    return x, y

def polar_velocity_to_cartesian(v_r, v_fi, r, fi):
    """Converts polar velocity to cartesian velocity."""
    sin = np.sin(fi)
    cos = np.cos(fi)
    ratio = v_fi * r
    vector_x = v_r * cos - ratio * sin
    vector_y = v_r * sin + ratio * cos
    velocity = np.sqrt(vector_x ** 2 + vector_y ** 2)
    return vector_x, vector_y, velocity

def polar_acceleration_to_cartesian(a_r, a_fi, v_r, v_fi, r, fi):

```

```

"""Converts polar acceleration to cartesian acceleration."""
cos = np.cos(fi)
sin = np.sin(fi)
v_ratio = 2 * v_r * v_fi
c_ratio = r * v_fi ** 2
a_ratio = r * a_fi
acceleration_x = a_r * cos - v_ratio * sin - c_ratio * cos - a_ratio * sin
acceleration_y = a_r * sin + v_ratio * cos - c_ratio * sin + a_ratio * cos
acceleration = np.sqrt(acceleration_x ** 2 + acceleration_y ** 2)
return acceleration_x, acceleration_y, acceleration

def calculate_tangential_acceleration(acceleration_x, acceleration_y, velocity_x, velocity_y):
    """Calculates tangential acceleration."""
    if velocity_x == velocity_y == 0:
        return 0, 0, 0
    velocity = np.sqrt(velocity_x ** 2 + velocity_y ** 2)
    tangential = (acceleration_x * velocity_x + acceleration_y * velocity_y) / velocity
    tangential_x = tangential * (velocity_x / velocity)
    tangential_y = tangential * (velocity_y / velocity)
    return tangential_x, tangential_y, tangential

def calculate_centripetal_acceleration(acceleration_x, acceleration_y, tangential_acceleration_x,
                                       tangential_acceleration_y):
    """Calculates centripetal acceleration."""
    centripetal_acceleration_x = acceleration_x - tangential_acceleration_x
    centripetal_acceleration_y = acceleration_y - tangential_acceleration_y
    centripetal_acceleration = np.sqrt(centripetal_acceleration_x ** 2 + centripetal_acceleration_y ** 2)
    return centripetal_acceleration_x, centripetal_acceleration_y, centripetal_acceleration

def calculate_curvature_radius(velocity, centripetal_acceleration,
                               centripetal_acceleration_x, centripetal_acceleration_y):
    """Calculates curvature radius."""
    if centripetal_acceleration == 0:
        return 0, 0, 0
    curvature_radius = velocity ** 2 / centripetal_acceleration
    ratio = curvature_radius / centripetal_acceleration
    curvature_radius_x = ratio * centripetal_acceleration_x
    curvature_radius_y = ratio * centripetal_acceleration_y
    return curvature_radius_x, curvature_radius_y, curvature_radius

if __name__ == '__main__':
    # --- Simulation settings ---

    t = sp.Symbol('t') # Time designation
    radius: sp.Expr | float | int = sp.cos(t) + 1 # Radius 'r' of particle point as function of time
    angle: sp.Expr | float | int = t * 5 / 4 # Angle 'fi' of particle point as function of time

    MAX_TIME = 30 # Maximum simulation time
    STEPS = 1000 # Number of steps

    FPS = 60 # Frames per second
    displaying_objects_switches = {
        'Position': True, # Display marker at point particle position.
        'Position vector': True, # Display vector from the origin to point particle position.
        'Trajectory': True, # Display curve based on set of points of point particle positions.
        'Curvature radius': True, # Display segment that is represents curvature radius.
        'Velocity vector': True, # Display vector of point particle velocity.
        'Total acceleration vector': True, # Display vector of point particle total acceleration.
        'Tangential acceleration vector': True, # Display vector of point particle tangential acceleration.
        'Centripetal acceleration vector': True # Display vector of point particle centripetal acceleration.
    }

    # --- Calculations ---

    velocity_radius = sp.diff(radius, t)
    velocity_angle = sp.diff(angle, t)

    acceleration_radius = sp.diff(velocity_radius, t)
    acceleration_angle = sp.diff(velocity_angle, t)

    time_points = np.linspace(start=0, stop=MAX_TIME, num=STEPS)

    radius_values = calculate_values(radius, time_points)
    angle_values = calculate_values(angle, time_points)

```

```

radius_velocity_values = calculate_values(velocity_radius, time_points)
angle_velocity_values = calculate_values(velocity_angle, time_points)

radius_acceleration_values = calculate_values(acceleration_radius, time_points)
angle_acceleration_values = calculate_values(acceleration_angle, time_points)

# --- Rendering ---
window = plt.figure()
region = window.add_subplot(1, 1, 1)
region.set_title("Вариант 20, Сергеев Владимир")

axes_limit = max(radius_values.max(), .1) * 3
region.set_xlim(-axes_limit, axes_limit)
region.set_ylim(-axes_limit, axes_limit)
# We will set the value to 'image' in order to comply with the established limits and avoid distortions.
region.axis('image')

all_objects = DisplayGroup()

plt.xlabel('x')
plt.ylabel('y')

plt.legend(handles=all_objects.return_plots_for_legend(), labels=all_objects.return_names_for_legend(),
           loc='lower left', fontsize='x-small')

def animate(i):
    x, y = polar_coordinates_to_cartesian(radius_values[i + 1], angle_values[i + 1])
    velocity_x, velocity_y, velocity = polar_velocity_to_cartesian(radius_velocity_values[i],
                                                                    angle_velocity_values[i],
                                                                    radius_values[i], angle_values[i])
    acceleration_x, acceleration_y, acceleration = polar_acceleration_to_cartesian(radius_acceleration_values[i],
                                                                                    angle_acceleration_values[i],
                                                                                    radius_velocity_values[i],
                                                                                    angle_velocity_values[i],
                                                                                    radius_values[i],
                                                                                    angle_values[i])
    tangential_x, tangential_y, _ = calculate_tangential_acceleration(acceleration_x, acceleration_y,
                                                                      velocity_x, velocity_y)
    centripetal_x, centripetal_y, centripetal = calculate_centripetal_acceleration(acceleration_x, acceleration_y,
                                                                                  tangential_x, tangential_y)
    curvature_radius_x, curvature_radius_y, _ = calculate_curvature_radius(velocity, centripetal, centripetal_x,
                                                                           centripetal_y)
    all_objects.update(x=x, y=y, velocity_x=velocity_x, velocity_y=velocity_y, acceleration_x=acceleration_x,
                      acceleration_y=acceleration_y, tangential_x=tangential_x, tangential_y=tangential_y,
                      centripetal_x=centripetal_x, centripetal_y=centripetal_y,
                      curvature_radius_x=curvature_radius_x, curvature_radius_y=curvature_radius_y)
    return all_objects.return_plots()

ani = FuncAnimation(window, animate, frames=STEPS, interval=round(1000 / FPS), repeat=False, blit=True)
plt.show()
print("Симуляция успешно завершена.")

```